

Zero-Shot Text-to-Image Generation

Aditya Ramesh¹ Mikhail Pavlov¹ Gabriel Goh¹ Scott Gray¹
Chelsea Voss¹ Alec Radford¹ Mark Chen¹ Ilya Sutskever¹

Abstract

Text-to-image generation has traditionally focused on finding better modeling assumptions for training on a fixed dataset. These assumptions might involve complex architectures, auxiliary losses, or side information such as object part labels or segmentation masks supplied during training. We describe a simple approach for this task based on a transformer that autoregressively models the text and image tokens as a single stream of data. With sufficient data and scale, our approach is competitive with previous domain-specific models when evaluated in a zero-shot fashion.

1. Introduction

Modern machine learning approaches to text to image synthesis started with the work of Mansimov et al. (2015), who showed that the DRAW Gregor et al. (2015) generative model, when extended to condition on image captions, could also generate novel visual scenes. Reed et al. (2016b) later demonstrated that using a generative adversarial network (Goodfellow et al., 2014), rather than a recurrent variational auto-encoder, improved image fidelity. Reed et al. (2016b) showed that this system could not only generate objects with recognizable properties, but also could *zero-shot* generalize to held-out categories.

Over the next few years, progress continued using a combination of methods. These include improving the generative model architecture with modifications like multi-scale generators (Zhang et al., 2017; 2018), integrating attention and auxiliary losses (Xu et al., 2018), and leveraging additional sources of conditioning information beyond just text (Reed et al., 2016a; Li et al., 2019; Koh et al., 2021).

Separately, Nguyen et al. (2017) propose an energy-based framework for conditional image generation that obtained a large improvement in sample quality relative to contem-



Figure 1. Comparison of original images (top) and reconstructions from the discrete VAE (bottom). The encoder downsamples the spatial resolution by a factor of 8. While details (e.g., the texture of the cat’s fur, the writing on the storefront, and the thin lines in the illustration) are sometimes lost or distorted, the main features of the image are still typically recognizable. We use a large vocabulary size of 8192 to mitigate the loss of information.

porary methods. Their approach can incorporate pretrained discriminative models, and they show that it is capable of performing text-to-image generation when applied to a captioning model pretrained on MS-COCO. More recently, Cho et al. (2020) also propose a method that involves optimizing the input to a pretrained cross-modal masked language model. While significant increases in visual fidelity have occurred as a result of the work since Mansimov et al. (2015), samples can still suffer from severe artifacts such as object distortion, illogical object placement, or unnatural blending of foreground and background elements.

Recent advances fueled by large-scale generative models suggest a possible route for further improvements. Specifically, when compute, model size, and data are scaled carefully, autoregressive transformers (Vaswani et al., 2017) have achieved impressive results in several domains such as text (Radford et al., 2019), images (Chen et al., 2020), and audio (Dhariwal et al., 2020).

By comparison, text-to-image generation has typically been evaluated on relatively small datasets such as MS-COCO and CUB-200 (Welinder et al., 2010). Could dataset size and

¹OpenAI, San Francisco, California, United States. Correspondence to: Aditya Ramesh <_@adityaramesh.com>.

Proceedings of the 37th International Conference on Machine Learning, Online, PMLR 139, 2020. Copyright 2020 by the author(s).



(a) a tapir made of accordion. (b) an illustration of a baby hedgehog in a christmas sweater walking a dog (c) a neon sign that reads “backprop”. a neon sign that reads “backprop”. backprop neon sign (d) the exact same cat on the top as a sketch on the bottom

Figure 2. With varying degrees of reliability, our model appears to be able to combine distinct concepts in plausible ways, create anthropomorphized versions of animals, render text, and perform some types of image-to-image translation.

model size be the limiting factor of current approaches? In this work, we demonstrate that training a 12-billion parameter autoregressive transformer on 250 million image-text pairs collected from the internet results in a flexible, high fidelity generative model of images controllable through natural language.

The resulting system achieves high quality image generation on the popular MS-COCO dataset *zero-shot*, without using any of the training labels. It is preferred over prior work trained on the dataset by human evaluators 90% of the time. We also find that it is able to perform complex tasks such as image-to-image translation at a rudimentary level. This previously required custom approaches (Isola et al., 2017), rather emerging as a capability of a single, large generative model.

2. Method

Our goal is to train a transformer (Vaswani et al., 2017) to autoregressively model the text and image tokens as a single stream of data. However, using pixels directly as image tokens would require an inordinate amount of memory for high-resolution images. Likelihood objectives tend to prioritize modeling short-range dependencies between pixels (Salimans et al., 2017), so much of the modeling capacity would be spent capturing high-frequency details instead of the low-frequency structure that makes objects visually recognizable to us.

We address these issues by using a two-stage training procedure, similar to (Oord et al., 2017; Razavi et al., 2019):

- **Stage 1.** We train a discrete variational autoencoder (dVAE)¹ to compress each 256×256 RGB image into a 32×32 grid of image tokens, each element of

which can assume 8192 possible values. This reduces the context size of the transformer by a factor of 192 without a large degradation in visual quality (see Figure 1).

- **Stage 2.** We concatenate up to 256 BPE-encoded text tokens with the $32 \times 32 = 1024$ image tokens, and train an autoregressive transformer to model the joint distribution over the text and image tokens.

The overall procedure can be viewed as maximizing the evidence lower bound (ELB) (Kingma & Welling, 2013; Rezende et al., 2014) on the joint likelihood of the model distribution over images x , captions y , and the tokens z for the encoded RGB image. We model this distribution using the factorization $p_{\theta,\psi}(x, y, z) = p_{\theta}(x | y, z)p_{\psi}(y, z)$, which yields the lower bound

$$\ln p_{\theta,\psi}(x, y) \geq \mathbb{E}_{z \sim q_{\phi}(z | x)} \left(\ln p_{\theta}(x | y, z) - \beta D_{\text{KL}}(q_{\phi}(y, z | x), p_{\psi}(y, z)) \right), \quad (1)$$

where:

- q_{ϕ} denotes the distribution over the 32×32 image tokens generated by the dVAE encoder given the RGB image x^2 ;
- p_{θ} denotes the distribution over the RGB images generated by the dVAE decoder given the image tokens; and
- p_{ψ} denotes the joint distribution over the text and image tokens modeled by the transformer.

Note that the bound only holds for $\beta = 1$, while in practice we find it helpful to use larger values (Higgins et al., 2016). The following subsections describe both stages in further detail.³

¹<https://github.com/openai/DALL-E>

²We assume that y is conditionally independent of x given z .

³In preliminary experiments on ImageNet (Deng et al., 2009),

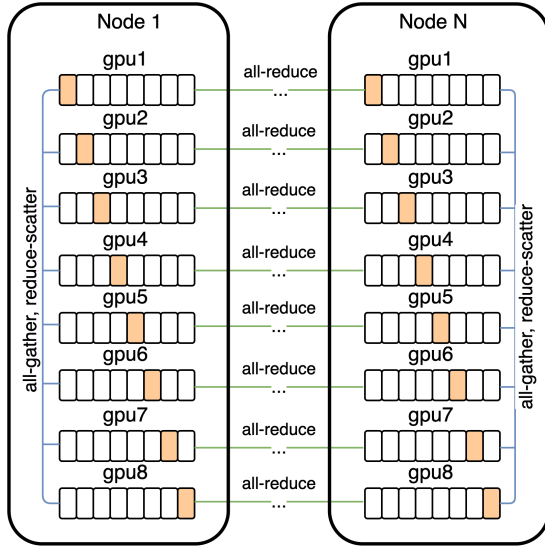


Figure 5. Communication patterns used for distributed training. Each parameter array in the model is sharded among the eight GPUs on each machine. During forward propagation, we prefetch the parameter shards for the next resblock (using all-gather) while computing the activations for the current resblock. To conserve memory, the parameter shards from the other GPUs are immediately discarded. Similarly, during backpropagation, we prefetch the parameter shards for the previous resblock while computing the activations and gradients for the current resblock. After all GPUs have computed the gradient with respect to an all-gathered parameter, the reduce-scatter operation leaves each GPU with only one slice – i.e., the gradient for its parameter shard, averaged over the eight GPUs.

no appreciable bearing on the results. We provide further details about the data collection process in Appendix C.

2.4. Mixed-Precision Training

To save GPU memory and increase throughput, most parameters, Adam moments, and activations are stored in 16-bit precision. We also use activation checkpointing and recompute the activations within the resblocks during the backward pass. Getting the model to train in 16-bit precision past one billion parameters, without diverging, was the most challenging part of this project.

We believe the root cause of this instability to be underflow in the 16-bit gradients. Appendix D presents a set of guidelines we developed to avoid underflow when training large-scale generative models. Here, we describe one of these guidelines: per-resblock gradient scaling.

Similar to prior work (Liu et al., 2020), we found that the norms of the activation gradients from the resblocks decrease monotonically as we move from the earlier resblocks

Effective Parameter Count	Compression Rank	Compression Rate
$2.8 \cdot 10^9$ ($d_{\text{model}} = 1920$)	512	$\approx 83\%$
$5.6 \cdot 10^9$ ($d_{\text{model}} = 2688$)	640	$\approx 85\%$
$12.0 \cdot 10^9$ ($d_{\text{model}} = 3968$)	896	$\approx 86\%$

Table 1. We show the relationship between model size and the minimum compression rank for the gradients (up to a multiple of 128) necessary to avoid a gap in the training loss during the first 10% of training. These results suggest that in our setting, we can achieve a compression rate of about 85%, independent of model size.

to the later ones.⁸ As the model is made deeper and wider, the true exponents of the activation gradients for later resblocks can fall below the minimum exponent of the 16-bit format. Consequently, they get rounded to zero, a phenomenon called *underflow*. We found that eliminating underflow allowed for stable training to convergence.

Standard loss scaling (Micikevicius et al., 2017) is able to avoid underflow when the range spanned by the smallest and largest activation gradients (in absolute value) fits within the exponent range of the 16-bit format. On NVIDIA V100 GPUs, this exponent range is specified by five bits. While this is sufficient for training vanilla language models of the same size, we found the range to be too small for the text-to-image model.

Our fix, which is shown in Figure 4, involves using a separate “gradient scale” for each resblock in the model. This can be seen as a practical alternative to a more general framework for mixed-precision training called Flexpoint (Köster et al., 2017), with the advantage that specialized GPU kernels are not required. We found that Sun et al. (2020) had independently developed similar procedure for training convolutional networks in 4-bit precision.

2.5. Distributed Optimization

Our 12-billion parameter model consumes about 24 GB of memory when stored in 16-bit precision, which exceeds the memory of a 16 GB NVIDIA V100 GPU. We address this using parameter sharding (Rajbhandari et al., 2019). As shown in Figure 5, parameter sharding allows us to almost completely hide the latency of the intra-machine communication by overlapping it with compute-intensive operations.

On the cluster used to train the model, the bandwidth between machines is much lower than the bandwidth among GPUs on the same machine. This makes the cost of the operation used to average the gradient among the machines (all-reduce) the main bottleneck during training. We were

⁸It is possible that better initialization schemes (Liu et al., 2020) might be able to avoid this, but we did not have success with alternative schemes in our experiments.